



NUS Orbital 2026

Project Group:

F4C_Opossum_6748



Prepared by:

WONG BO XI FILBERT
LI JIANXI

Table of Contents

1. Executive Summary	3
2. Project Vision & Motivation	3
2.1. Mission Statement	3
2.2. Academic Motivation	3
3. Target Audience & User Stories	3
4. Core Game Features (Milestone 1)	4
4.1. Basic 2D Engine setup	4
4.2. Entity Spawning System	4
4.3. Combat & Movement AI	6
4.4. Modular Health & UI System	8
4.5. Dynamic Camera System	8
5. Future Roadmap (Milestones 2 & 3)	9
5.1. Milestone 2 Priorities	9
5.2. Milestone 3 Priorities	10
6. Modern Technical Stack	10
7. Software Engineering Rigour	10
8. Methodology & Quality Assurance	11
9. Conclusion for Milestone 1	11
10. Source Code Documentation & Engineering Notes	11
11. Milestone 2: Expanding Complexity and Polishing	12
11.1. Art Integration, Animation, and Asset Challenges	12
11.1.1. Pixel Per Unit (PPU) Inconsistencies:	12
11.1.2. Pivot Point Misalignments:	12
11.1.3. Animator Integration & Animation Events:	13
11.1.4. Lack of Free Assets and 2D Sprites:	14
11.2. Economic System	15
11.3. Advanced Unit Formation and Movement Animation	16
11.4. Main Menu and Environmental Effects	17
11.5. Scene Transition System	19
11.6. Audio Management System	20
11.7. User Interface Refinement	20
11.8. Pause Menu System	22
11.9. Turret Construction and Defence System	23
11.10. Unity Animator State Machine	24
11.11. Version Control with Unity	26
11.12. Conclusion for Milestone 2	26
11.13. Future Roadmap for Milestone 3	27

1. Executive Summary

This document serves as the official Milestone report for Project F4C Opossum, developed for the NUS Orbital 2026 programme. Aiming for the Apollo 11 level of achievement, our team is engineering a highly scalable, robust 2D side-scrolling strategy game using the Unity Engine and C#. This first milestone demonstrates a fully functional Proof of Concept (PoC) that encompasses the core game loop: resource economy, automated entity spawning, AI-driven adversarial generation, collision-based combat, and a definitive win/loss state via base destruction.

2. Project Vision & Motivation

2.1. Mission Statement

Inspired by the timeless mechanics of the classic flash game Age of War, F4C Opossum modernises the tug-of-war base defence genre. The game challenges players to balance a constantly ticking resource economy with tactical unit deployment to overwhelm a dynamic enemy AI.

2.2. Academic Motivation

As Computer Science undergraduates, we see this project as a deliberate push beyond standard university curricula. Building a game from scratch requires mastering complex, real-time software architectures, UI/UX interaction paradigms, and optimising intricate game loops. Targeting Apollo 11 requires us not only to deliver a fun game but also to adhere strictly to industry-standard software engineering practices, ensuring our codebase is clean, modular, and extensible.

3. Target Audience & User Stories

To ensure our development remains user-centric, we have defined four core player personas:

- **The Strategist:** "As a strategist, I want to balance my gold economy between spawning cheap frontline units and saving for expensive ranged units, so I can optimally counter the enemy's formation."
- **The Competitor:** "As a competitive player, I want the enemy AI to spawn units autonomously and unpredictably, so the game remains challenging and forces me to adapt my strategy on the fly."
- **The Explorer:** "As an explorer, I want to see visual feedback when my units clash with the enemy, so I can clearly understand the combat mechanics and health systems."

- **The Casual Gamer:** "As a casual player, I want an intuitive user interface with clear buttons and resource counters, so I can understand how to play within the first ten seconds of launching the game."

4. Core Game Features (Milestone 1)

To build a solid foundation for F4C Opossum, our initial milestones focused on establishing the core gameplay loop, entity interactions, and robust C# scripts. Currently, our Milestone 1 submission focuses on building the technical foundation necessary for the future gameplay systems. The current prototype demonstrates:

- A functioning 2D environment
- Unit spawning systems
- Movement and collision systems
- Base health management
- Combat interactions
- Dynamic HP bar rendering

These features help to establish the minimum viable gameplay loop required for future expansion.

4.1. Basic 2D Engine setup

To start the project, we had to set up the overview of how the gamemap would look, as well as the layers we had to consider when building the map. For example, different layers like (foreground, default, background), as well as layers for unit tagging to differentiate between allied units and enemy units.

As we are planning to build an Age of War game based on a 2D side-scrolling game, we decided to utilise the Unity Universal 2D template.

4.2. Entity Spawning System

Since Age of War is a 2D Player versus Enemy game that requires user input to spawn units against AI/bot enemies, we needed to create spawning scripts for both player-controlled and enemy-controlled units.

Current Functionality:

- Dynamic runtime instantiation
- Separate spawn points
- Independent player/enemy spawning systems

Scripts Created for Spawning System:

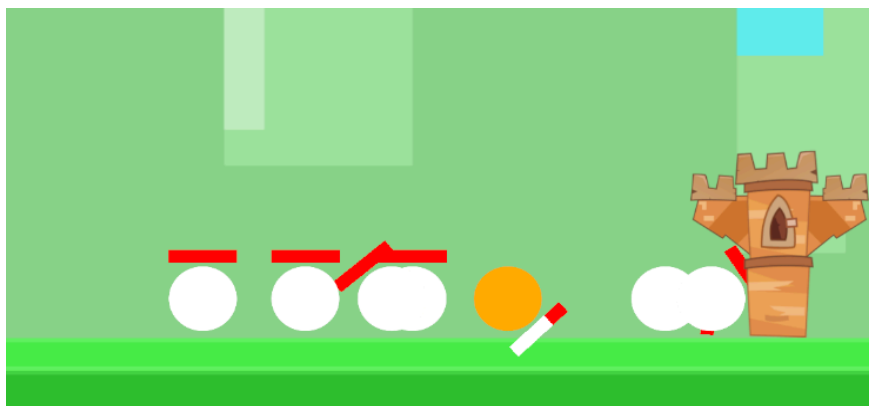
- PlayerSpawner.cs
- EnemySpawner.cs

Design Configurations:

- Player unit prefabs are loaded dynamically from a "Resources/PlayerUnits" folder at startup.
- The player (user) can deploy units by pressing the "Z" key.
- Originally, a 0.5-second cooldown was enforced on player spawning to prevent units from jittering down and improperly stacking. However, since we had factored in the idea that the unit stacking is alright for allies only at the base, the original jittering issue has been resolved, where the cooldown now is to simulate the time it takes to "train" up the unit.
- Enemy units are similarly loaded from a "Resources/EnemyUnits" folder.
- Enemies currently spawn automatically on a 25-second timer.
- Both spawners actively monitor the battlefield and will halt all spawning if their respective base's HealthSystem is destroyed.
- A sequential spawnOrder tracker tags every instantiated unit, allowing the game to distinguish older units from newer ones. This feature prevents unit stacking bugs, as stacked units now know which unit should move first.

Challenges faced during the building process of this feature:

One issue encountered was unit overlap during spawning. This occasionally caused units to collide incorrectly or become stuck during movement. This was partially resolved by implementing collision checking and movement spacing between allied units. The image below shows the units colliding during testing of the collision mechanics. In the previous version, the enemy unit spawn was too close to the map, causing enemy units to spawn on "ally" units and creating this undesired bug.



Design Decisions and Future Considerations:

We chose to create two separate scripts, one for ally spawning and one for enemy spawning, since we know that enemy spawning is initiated by the Computer code, so there is no need to add features that read user input to spawn units. On the other hand, since the player units are spawned not based on time but by user input, we need to factor the reading of user inputs into the PlayerSpawner.cs script.

However, future considerations include adding features like monetary tracking so that units can only be spawned when the player/enemy has enough money to deploy them, as well as creating buttons to deploy units instead of using keycap values for players. That way, the user can see what the unit they are deploying looks like before choosing to deploy it.

4.3. Combat & Movement AI

The combat and Movement feature allows units to move automatically toward enemy targets and engage in combat once enemies enter range. The units are also able to do smart switching between enemy units and the enemy base, where they will switch targets to the earliest spawned unit to attack if they were initially attacking the enemy base when this enemy unit was spawned.

Current Functionality:

- Automatic movement
- Enemy detection
- Attack cooldown systems
- Damage application
- Base destruction logic

Scripts Created for Unit Movement and HP interactions:

- UnitMove.cs
- HealthSystem.cs

How the Combat System works:

- Units begin in an undeployed state and must fall to physically hit the ground layer (detected via Physics2D.OverlapCircle) before they can begin marching.
- Movement direction is determined dynamically by Unity tags; players move right (positive X direction), while enemies move left (negative X direction).

- **Ally Collision:** Units utilise 2D Raycasts (Physics2D.RaycastAll) filtered by layer masks to detect allies directly ahead of them. A unit will stop and wait if the ally in front of it has an older spawnOrder, effectively preventing units from clipping through or passing each other.
- **Targeting:** Units use dynamic bounding boxes (Physics2D.OverlapBoxAll) to detect targets within their specific attack range. The AI scans the overlapping zone and prioritises attacking the oldest spawned enemy unit first. If no enemy units are alive, the AI seamlessly switches its target to the enemy base.
- Combat operates on a 1-second attack cooldown. (This is a placeholder value which can be customised for different units in the future)
- Units deal their predefined damage values directly to the target's HealthSystem once the cooldown completes.

Design Decisions and Future Considerations:

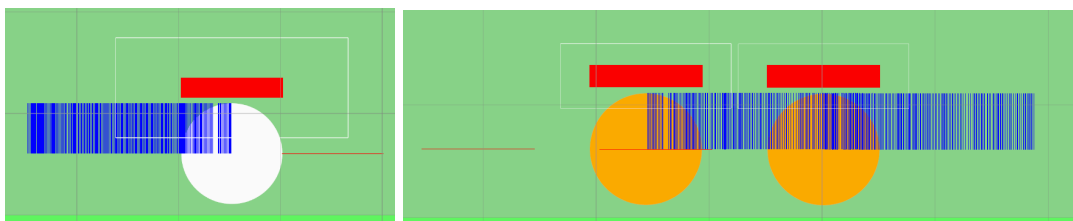
The current combat system design has multiple functions that handle the following:

- Movement logic
- Health management
- Attack behaviour

However, all these functions are currently in a single file, UnitMove.cs. As such, future considerations include breaking these functions into new files to keep them clean and more meaningful.

Challenges faced during the building process of this feature:

Throughout the process of creating, there were a lot of trials and errors in fine-tuning the unit collision detection for both allies (RayCast) and enemies (BoxAll) to ensure no unintended behaviour would occur during gameplay testing. The image below shows the debugging steps and trackers we place to see how the combat collision detection system is used, with the red line indicating the unit's "sight" in front to determine whether the unit in front is an ally or a foe. The blue line is all objects it was colliding with, which we found out was because it was tracking the collision with itself as well.



4.4. Modular Health & UI System

Following a Dynamic HP bars template from YouTube tutorials, we implemented it for both bases and combat units. We also fine-tuned the template, allowing the Health Points bar to drop vertically or horizontally so the system accommodates both orientations.

Current Functionality:

- Real-time HP updates
- World-space UI tracking
- Visual combat feedback

Scripts Created for Health Bar UI:

- HealthBarUI.cs
- HealthSystem.cs

How the Health and UI System works:

- We developed a modular HealthSystem component to manage hit points, with a default maximum of 100 HP (temporary value). Currently, bases have 500 HP, enemy units have 100 HP, and ally units have 150 HP.
- The system takes damage, clamps health values to prevent bugs, and automatically destroys the GameObject when HP reaches zero.
- The system is decoupled from the HealthBarUI script, which visually represents health percentages by adjusting a RectTransform's size.
- The UI supports horizontal orientations for standard units and vertical orientations for the bases.

Challenges faced during the building process of this feature:

Initially, a challenge was how to get the HealthBar to move with the units rather than remain static. since the tutorial did not really cover assigning the health bar to a unit. After searching for a bit, the fix was to put the health bar, in this case the “Canvas UI”, as a child of the UI; that way, when the unit moves, the health bar moves along with the unit.

4.5. Dynamic Camera System

Originally, we created the Gamemap layout to fit only the current camera point of View, with the entire map situated nicely within the camera boundaries. However, during testing, we felt that the “runway” for units to move was too short. As a result, to expand the map, we also need to ensure that the camera can move freely across the map without needing to be moved by following a player unit (as

covered in the Mission Control 2 Webinar), but by following the user's mouse cursor instead.

Therefore, this feature is designed to let a camera controller keep the battlefield visible throughout gameplay, allowing the user to see different parts of the map.

Scripts Created for Health Bar UI:

- CameraController.cs

How the Camera Controller movement works:

- Players can pan the camera horizontally by moving the mouse cursor within 50 pixels of the edges of the screen.
- The camera's movement is clamped to predefined background boundaries to keep the focus on the battlefield.
- Users can utilize the mouse scroll wheel to zoom in and out.
- The zoom, which utilises a mouse scroll wheel, is clamped between a minimum and maximum value, and the camera is mathematically anchored to ground level, so zooming out reveals more of the map rather than exposing underground areas.

5. Future Roadmap (Milestones 2 & 3)

With the MVP architecture secured, development will now pivot to expanding complexity, balance, and visual fidelity.

5.1. Milestone 2 Priorities

- **Monetary Systems with Buttons to Spawn Units:** Implement a currency tracker that tracks the player's current balance to identify which units the player can summon and which units the player is unable to summon. This includes earning money by defeating enemy troops, having a base starting amount, and potentially having a "turret" that, instead of attacking enemy units, helps earn "coins."
- **Art Integration:** Replace placeholder geometry with fully animated 2D sprites (idle, walk, attack, and death animations).
- **Implementation of Turret Defence:** Since our Bases right now are quite empty, with nothing to protect them, we need to implement slots that can be unlocked via the currency, as well as turrets that can be placed on the unlocked slots to deal damage to enemy when within their range or do passive abilities, like gaining currency.

5.2. Milestone 3 Priorities

- **Special Abilities:** Integrate cooldown-based global abilities (e.g., calling down a meteor strike on the battlefield).
- **Advanced AI:** Upgrade the EnemySpawner script to read the player's unit composition and intelligently spawn counter-units, rather than relying on randomised timers.
- **Audio Engineering:** Add background tracks, UI interaction clicks, and combat sound effects.
- **Polish & CI/CD:** Finalise game balance, conduct extensive bug testing, and potentially set up automated web-GL deployment via GitHub Actions.

6. Modern Technical Stack

F4C Opossum was built using industry-standard tools to ensure a high-quality end product.

- **Development Engine:** Unity (C#) was chosen as our primary engine because it is the industry standard for handling high-performance 2D game loops and physics calculations.
- **Art & Assets:** We utilised Aseprite as our precision tool for creating custom 2D pixel art assets and animations, supplemented by consistent, professional visuals sourced from Kenney.nl.
- **Project Management:** We used Git & GitHub for version control (employing a feature-branch workflow with peer reviews), and Kanban boards for agile sprint tracking to maintain transparent progress.

7. Software Engineering Rigour

To keep our codebase scalable and maintainable, we applied several established design patterns and strict encapsulation:

- **State Pattern:** Applied to manage overall game phases (Menu, Playing, Paused) as well as the distinct behavioural states of individual units on the battlefield.
- **Observer Pattern:** Used extensively to decouple the UI elements (such as health bars managed by HealthBarUI and resource counters) from the underlying core game logic (managed by HealthSystem), ensuring that visual updates do not bottleneck game performance.

- **OOP Excellence & Encapsulation:** We enforced strict object-oriented programming principles across all scripts. For example, critical tracking variables like `spawnOrder` are strictly encapsulated with the `[SerializeField]` attribute, making them inaccessible to other scripts while still visible in the Unity Inspector for debugging.

8. Methodology & Quality Assurance

Consistent iterative development and testing were paramount to our success.

- **Agile Sprints:** We operated in two-week development cycles, enabling continuous, iterative feedback and rapid pivoting when mechanics required rebalancing.
- **CI/CD Workflow:** We strictly enforced a feature-branching strategy with mandatory Pull Requests (PRs), which effectively prevented regressions from breaking our main branch.
- **Testing Strategy:** We dedicated significant time to extensive bug testing and code refactoring, resulting in a highly polished user experience.

9. Conclusion for Milestone 1

Our Milestone 1 stage successfully establishes the project's technical foundation through a functional combat prototype and modular gameplay systems. The current implementation demonstrates the project's feasibility and provides a scalable architecture for future gameplay extensions, such as AI progression systems, economy management, and special abilities.

10. Source Code Documentation & Engineering Notes

The GitHub repository contains detailed inline code comments and engineering notes explaining key gameplay systems, implementation decisions, and debugging considerations throughout development.

These comments were added to improve:

- Maintainability
- Scalability
- Readability
- Debugging efficiency

Key systems documented include:

- Combat and collision systems
- Movement state handling
- Raycast detection logic
- Spawn management systems
- HP synchronisation systems

As for where the relevant scripts covered in this document can be found, they are under the “Assets/Scripts/” folder.

GitHub Repo Link:

https://github.com/Filbertabulate/F4C_Opossum_6748

Video Gameplay for Milestone 1:

<https://drive.google.com/file/d/1kpUUDA6sRINWCUp549kR7V0Ms-xXScX/view?usp=sharing>

11. Milestone 2: Expanding Complexity and Polishing

Building upon the MVP architecture secured in Milestone 1, development for Milestone 2 pivoted toward expanding game complexity, refining visual fidelity, and implementing the planned economy and defence systems. This phase introduced significant challenges in asset integration and required robust debugging of movement logic to maintain a seamless gameplay loop.

11.1. Art Integration, Animation, and Asset Challenges

A major priority for this milestone was replacing placeholder geometry with fully animated 2D sprites (idle, walk, and attack animations). Sourcing and integrating assets from different artists also introduced several technical and visual inconsistencies that required systematic debugging.

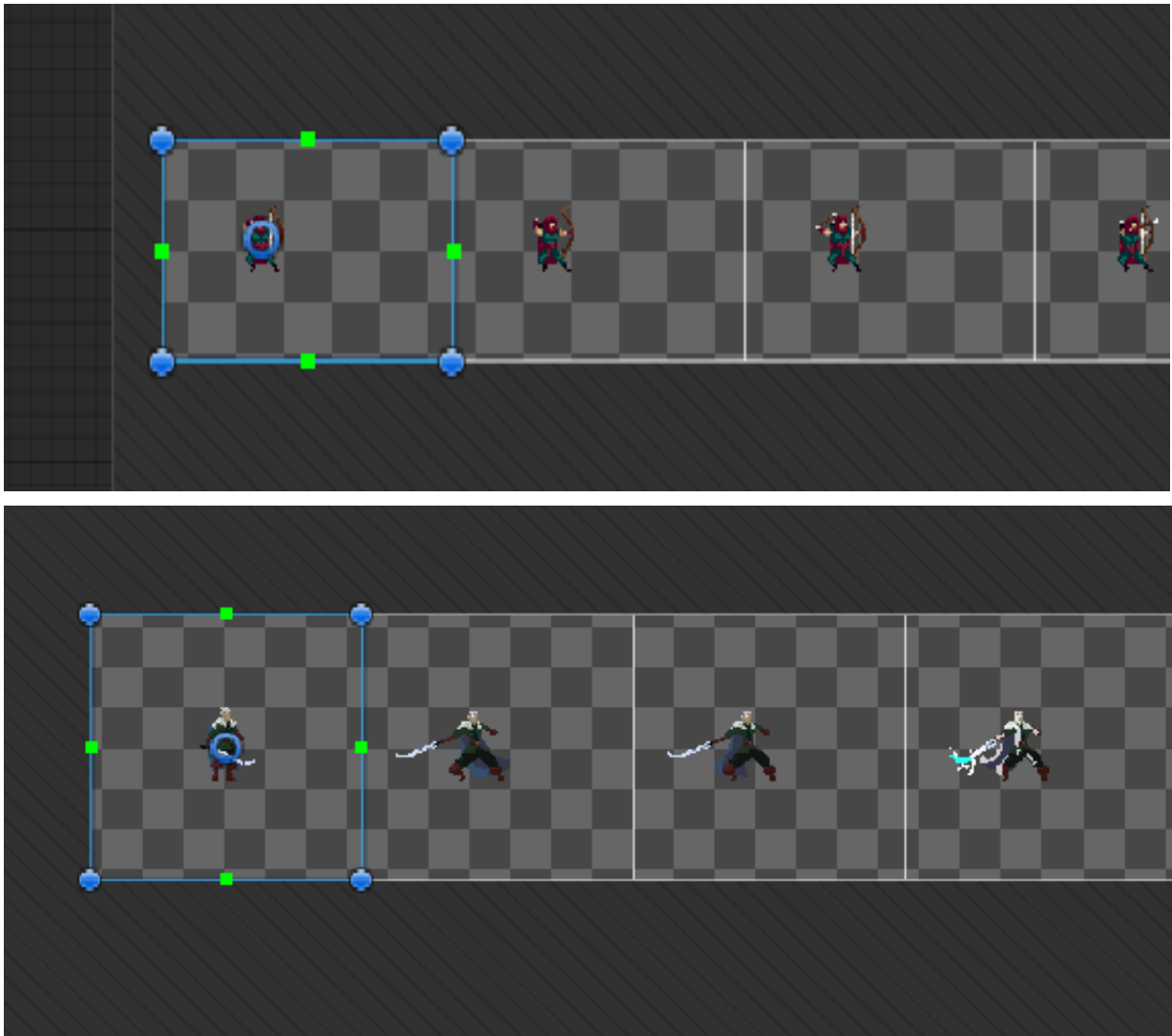
11.1.1. Pixel Per Unit (PPU) Inconsistencies:

Assets sourced from different creators often had vastly different pixel densities. This caused units to spawn at drastically different scales in the Unity engine. We standardised the PPU across all imported textures to ensure consistent scaling.

11.1.2. Pivot Point Misalignments:

Different artists utilise different centre anchor points for their sprites (e.g., bottom-centre vs true centre). When instantiating units dynamically, this discrepancy caused certain units to spawn slightly underground or float above the ground layer.

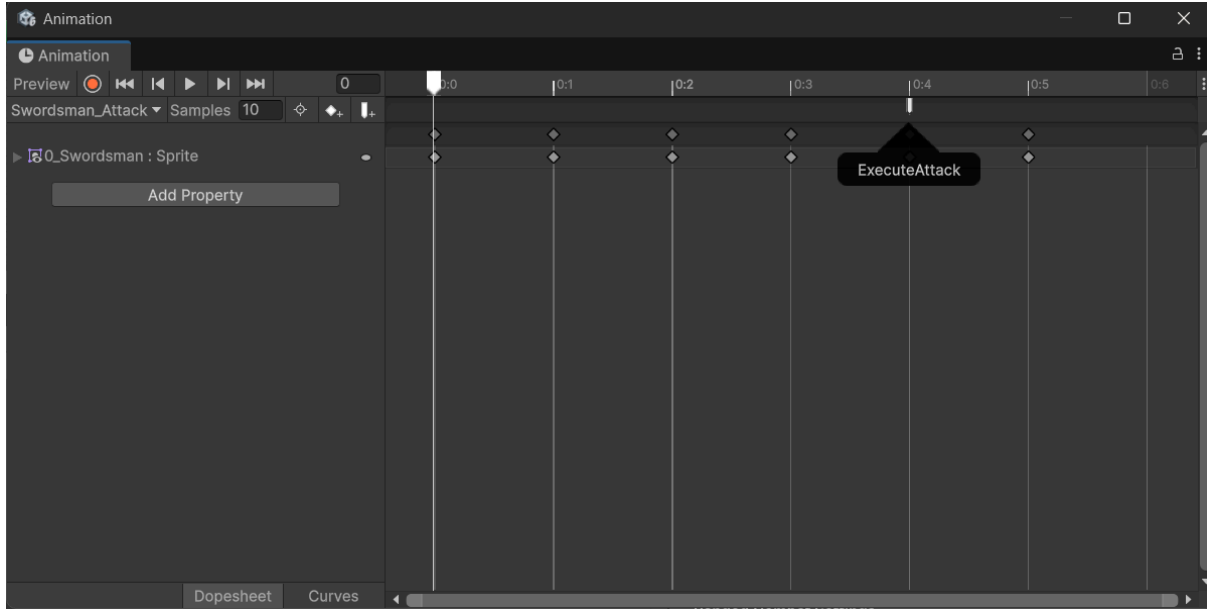
We resolved this by meticulously standardising the pivot points in Unity's Sprite Editor for all unit frames.



Note: On the top is the player Archer unit attack animation frame by frame, while on the bottom is the player magic knight unit attack animation frame by frame

11.1.3. Animator Integration & Animation Events:

To ensure combat visual feedback aligned accurately with the underlying damage calculations, we heavily utilised Unity's Animator component. A significant challenge was ensuring units did not deal damage before their attack animation physically connected with the enemy. To resolve this issue, we decoupled the attack timer from the direct damage application. Instead, the timer triggers the `AttackTrigger` in the Animator, and an Animation Event embedded directly into the sprite's attack frame calls the `ExecuteAttack()` method.



Note: Unity Animation panel that allows additions of event triggers at a certain frame of the animation to make it look smooth.

11.1.4. Lack of Free Assets and 2D Sprites:

When embarking on this project, we initially underestimated the sheer volume of artistic resources required to bring a game to life. While writing the backend logic and C# scripts presented its own deep technical challenges, establishing a compelling visual identity proved to be one of our most significant bottlenecks.

One of the most persistent issues we faced was the scarcity of high-quality, free-to-use digital assets and 2D sprites. Because F4C Opossum is an educational, zero-budget project intended to be entirely free-to-play, purchasing premium asset packs was simply out of the question.

Consequently, we had to rely heavily on open-source platforms. We spent a tremendous amount of time and energy scouring the internet just to find characters, enemies, and environments that actually looked like they belonged in the same universe. Trying to stitch together artwork created by completely different artists, each utilising their own unique colour palettes, shading techniques, and pixel densities, was an exhausting process. It took painstaking effort to ensure our game did not end up looking like a disjointed, visual patchwork, and required us to heavily edit and standardise the assets we did manage to find.

We also experimented with AI image generation tools to create certain pixel-art assets when suitable free alternatives are less desirable. However, the AI-generated

sprites rarely integrate seamlessly into our game. This is because they often varied in proportions, animation style, colour palette, and pixel density, making them visually inconsistent with the rest of our assets.

Moreover, even when the AI generate a sprite sheet, we have to take that sprite sheet to a free photo editing platform like Photopea to crop out each image, making sure that each background is transparent manually, before needing to stitch them back up again from individual PNG images. Although this workflow eventually produced somewhat usable assets, the repetitive manual editing significantly increased the development time for even a single animated character.

Ultimately, navigating this logistical nightmare gave us a profound, newfound appreciation for the game development industry at large. Experiencing firsthand the staggering amount of effort it takes just to source, format, and animate a handful of 2D pixel-art sprites for a student prototype really highlighted the monumental labour required by entire departments of conceptual artists, riggers, and technical animators to develop visually breathtaking AAA titles like Elden Ring.

11.2. Economic System

Current Functionality:

To deepen the strategic layer, we implemented the planned monetary system, requiring players to balance resource generation against unit deployment costs. The UI was completely overhauled to display current Gold, Experience Points (EXP), and specific unit deployment buttons.

Scripts Modified

- PlayerSpawner.cs

How the Economic System Works

Currently, a passive income generator is being used in the Player Spawner script, providing a baseline economy. The UI buttons were refactored to pass a specific `unitIndex` parameter, ensuring players can make tactical choices rather than relying on randomised spawning.

- 50 Gold at the start, gain 5 Gold per game second
- 0 EXP at the start, gain 1 EXP per game second

11.3. Advanced Unit Formation and Movement Animation

Current Functionality:

Building upon the combat system developed in Milestone 1, the movement animation was significantly enhanced to allow units to maintain smooth marching formations. Rather than simply stopping whenever another allied unit was detected in front, units are now capable of dynamically matching the movement speed of the unit they are following. This prevents large formations from constantly stopping and starting, resulting in much smoother gameplay and more natural-looking marching animations.

Scripts Modified

- UnitMove.cs
- EnemySpawner.cs

How the Movement System Works

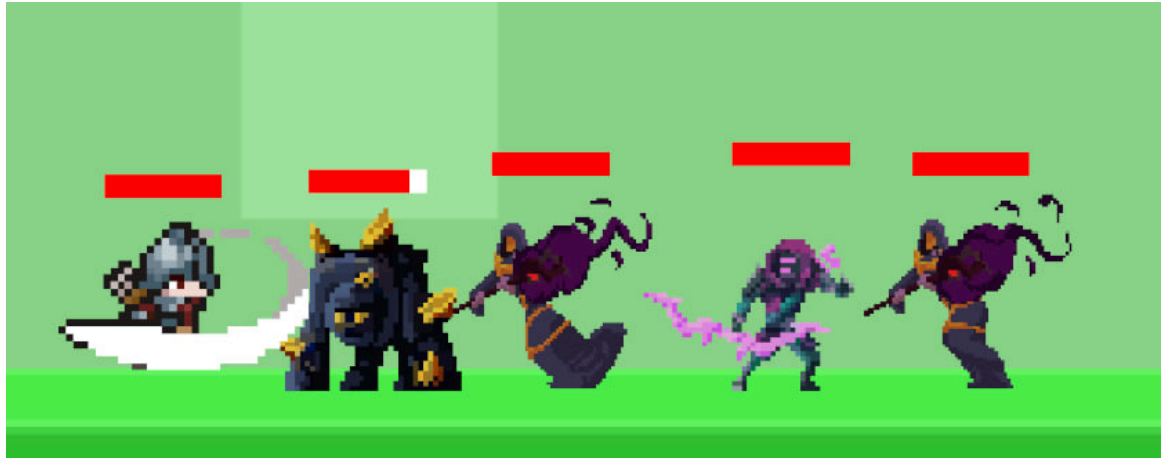
Originally, the previous implementation relied only on an ally detection raycast positioned in front of each unit. Whenever another allied unit entered this detection zone, the trailing unit would immediately stop until the path became clear.

The updated implementation introduces a second tracking system that continuously monitors the ally directly ahead. Once a unit resumes movement, it compares its own movement speed against the currently tracked ally (the one directly in front of the current unit) and temporarily adopts the slower movement speed whenever necessary. By synchronising movement speeds instead of repeatedly stopping, units can maintain a consistent pace while moving together.

This behaviour also allows attack animations and walking animations to remain synchronised across large groups of units, significantly improving the overall visual quality of combat.

Challenges faced during the building process of this feature:

One major issue encountered was incorrect ally detection. During the initial implementation, units only relied on a single forward-facing detection ray. As units constantly entered and exited this raycast, they frequently locked onto the wrong ally, causing units to repeatedly stop, accelerate, and / or switch targets. This resulted in noticeable animation glitches and jittering movement.



Note: As shown, the enemy units' fire mage is still constantly running behind the golem, even though the enemy golem is currently engaging in battle and is not moving any more.

To resolve this issue, as mentioned, a separate ally tracking system was implemented alongside the original forward detection logic. The forward raycast determines when a unit should stop, while the tracking system continuously follows the specific ally directly ahead, allowing smooth speed synchronisation without unnecessary movement interruptions.

11.4. Main Menu and Environmental Effects

Current Functionality:

To improve the player's first impression of the game, a fully interactive main menu was created. The menu incorporates mouse-driven parallax effects, animated environmental elements, and a continuously animated hero character to create a more dynamic title screen.

Scripts Created

- MenuParallaxBehaviour.cs
- Animator Controllers (Fall.anim)

How the Main Menu System Works

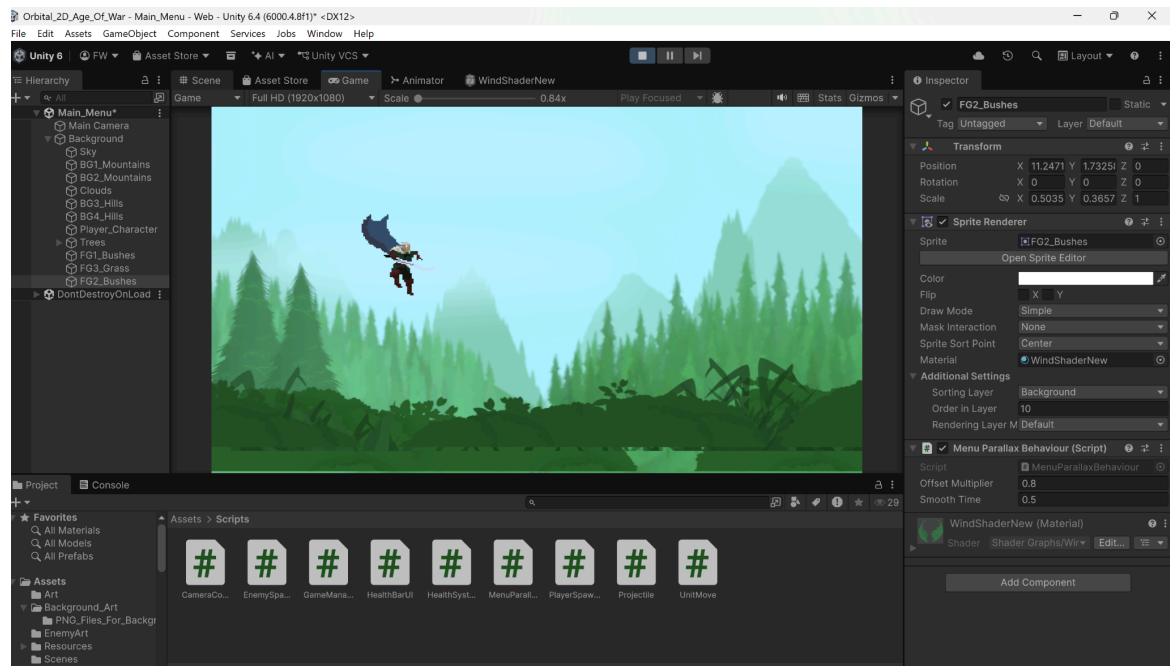
With this implementation, the foreground and background objects respond to the user's mouse position using smooth interpolation, creating a layered parallax effect that gives the menu additional depth. Environmental assets (By [Raycastly](#)), such as trees and grass, are animated simultaneously using shader-based wind effects, all while the hero character continuously loops a falling animation.

Challenges faced during the building process of this feature:

Achieving a natural-looking environment required a considerable amount of fine-tuning. Initially, the cape animation of the hero moved at a noticeably different speed from the surrounding wind animations applied to the trees and grass. Although each animation functioned independently, the mismatch reduced the overall realism of the scene.

After multiple trial and error, the playback speed of the character's animation was adjusted to better match the movement speed of the environmental wind shader, creating a much more visually cohesive main menu.

Another issue involved the foreground grass used in the parallax system. Large mouse movements caused the grass layer to shift too far, exposing the background beneath the map. This was resolved by enlarging the foreground grass sprite while simultaneously reducing the overall parallax movement multiplier.



Note: The error shown above is the grass being exposed when the mouse movement is too large, causing it to be exposed. (A considerable amount of time and effort was spent on placing each tree, grass and all other components in this current backdrop).

11.5. Scene Transition System

Current Functionality:

To create smoother transitions between gameplay scenes, both cross-fade and circular fade (Not being used at the moment, but is a good thing to have for potential future implementations) transition effects were implemented. These transitions are utilised throughout the game when we are switching between the gameplay scene and the main menu scene.

Scripts Created

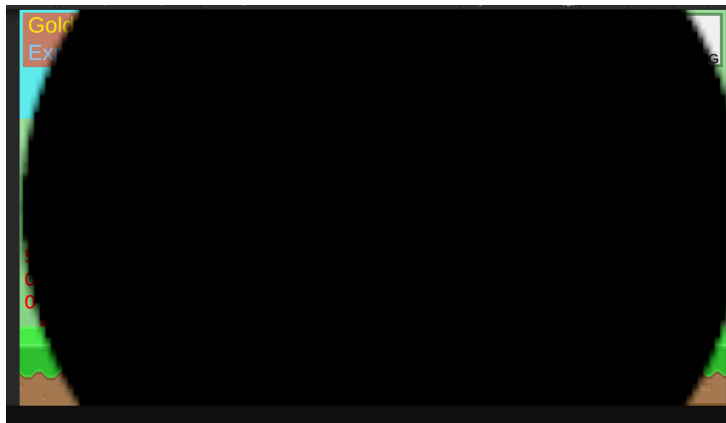
- LevelManager.cs
- SceneTransition.cs
- CrossFade.cs
- CircleWipe.cs

How the Transition System Works

Scene loading is controlled through a central LevelManager, which triggers different transition animations depending on the destination scene. The transition animation plays before asynchronously loading the next scene, resulting in a seamless visual experience. Moreover, from the main menu screen, when play is clicked, the transition loads with a progress task bar being shown to showcase the progress between transiting the main menu to the gameplay mode.

Challenges faced during the building process of this feature:

One issue encountered was regarding the circular fade transition. Initially, the expanding circle never completely covered the screen because its size was calculated relative to the world space rather than the UI Canvas.



Note: Circle wipe transition does not fully cover the screen at first

How we managed to fix this issue is by increasing the maximum radius of the transition sprite while anchoring the animation relative to the Canvas itself. This ensured that every transition fully covered the display, regardless of screen resolution.

11.6. Audio Management System

Current Functionality:

A global audio management system was implemented to control background music, button click effects, button hover effects, and user-configurable volume settings across every scene in the game.

Scripts Created

- AudioSettingsUI.cs
- MusicLibrary.cs
- MusicManager.cs
- SoundLibrary.cs
- SoundManager.cs

How the Audio System Works

A persistent AudioManager, i.e., the MusicManager and SoundManager, is maintained across scene changes using Unity's singleton pattern. Between scenes, if there is a change in audio volume, it would tie back to the Manager instance to change volume, background music for MusicManager and SFX music for Sound Manager.

Challenges faced during the building process of this feature:

The main challenge was trying to enforce the background music to be constantly played after changing scenes. Without setting up a persistent audio manager, Unity recreated new AudioSources every time a scene loaded, causing the music to restart repeatedly. Therefore, I had configured the game to have the audio manager at the main menu screen, since this is the very first scene that will be loaded for the users.

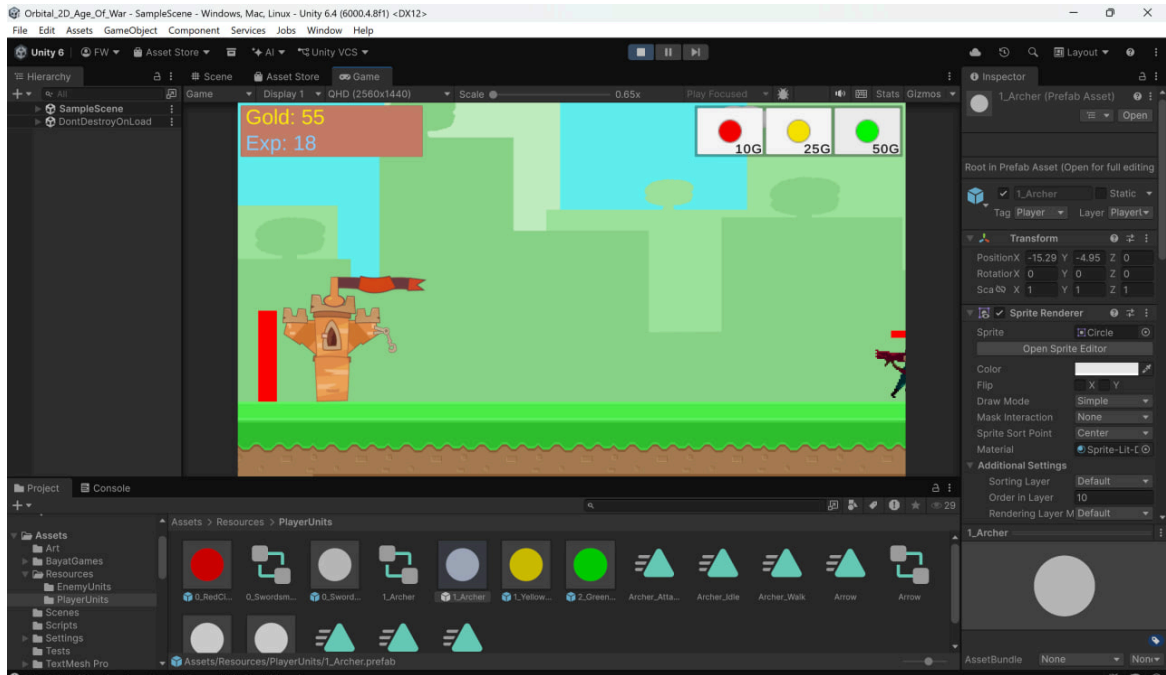
11.7. User Interface Refinement

Current Functionality:

To improve visual clarity, the interface for purchasing units, unlocking turret slots, and displaying player resources was redesigned using custom pixel-art assets (Mainly AI generated since we could not really find UI templates that match what

we want). New icons for Gold, Experience Points, turret slots, and deployment buttons, castle sprites were introduced to better match the overall medieval aesthetic of the game.

Before UI Refinement:



After UI Refinement



Challenges faced during the building process of this feature:

The main issue was just finding UI assets that showcase the theme we are going for. Considerable time was spent editing, resizing, and standardising these assets to create a consistent user interface.

11.8. Pause Menu System

Current Functionality:

A pause system was implemented, allowing players to pause gameplay, adjust settings, resume the game, or return to the main menu.

Scripts Created

- PauseManager.cs

How the Pause System Works

Activating the pause menu sets Unity's Time.timeScale to zero while displaying a full-screen overlay containing navigation buttons and audio settings.

Challenges faced during the building process of this feature:

Initially, the pause menu failed to properly cover the gameplay scene. Certain interface elements also disappeared whenever the game window was resized.



Note: As shown, the grass and tower, as well as the HP bar, are not being covered by the translucent background even though the game pause menu is activated.

Researching online, it turns out the main issue was that the Canvas had been configured using Screen Space - Camera mode. Migrating the UI to Screen Space - Overlay mode resolved the rendering order issues. Following this migration, all interface elements were manually repositioned and resized to ensure correct behaviour across different screen resolutions.

11.9. Turret Construction and Defence System

Current Functionality:

A modular turret construction system was developed, allowing players to first unlock turret scaffolding (using EXP) before constructing fully operational defensive towers (Using Gold). Once deployed, turrets automatically detect nearby enemies and fire projectiles whenever targets enter their attack range.

Scripts Created

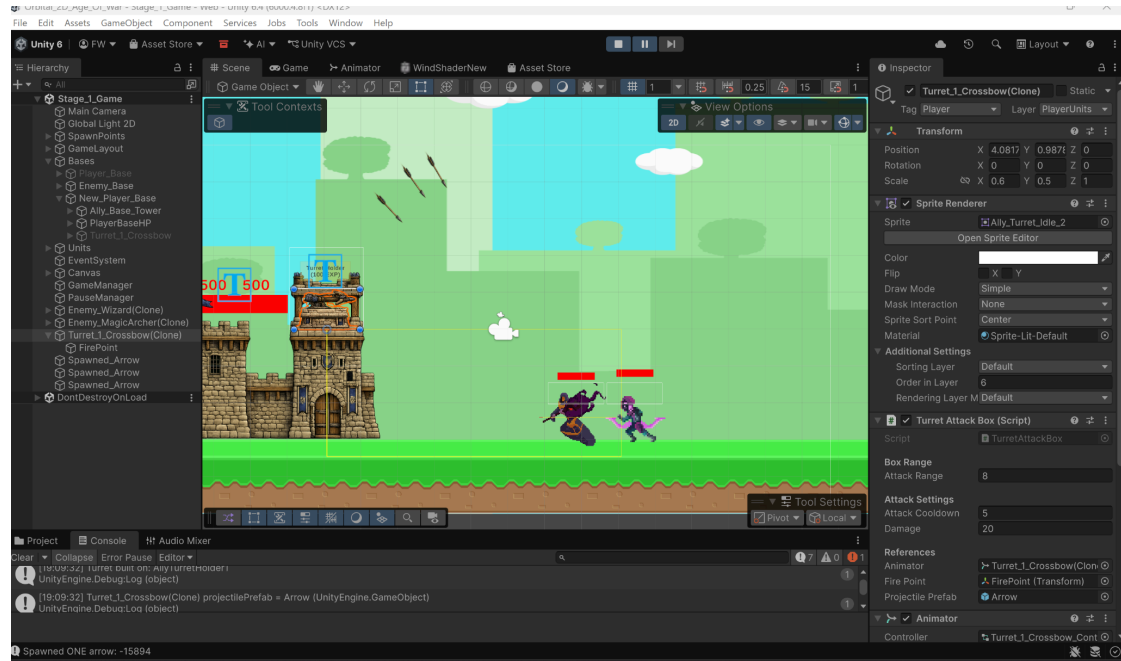
- TurretSlotUI.cs
- TurretAttackBox.cs
- TurretArcProjectile.cs

How the Main Menu System Works

Each turret continuously scans its attack range for enemy units. Upon detecting a valid target, the turret rotates towards the enemy, plays its attack animation, and launches a projectile towards the selected target. The scanning method uses a BoxAll method to find the earliest spawned enemy that enters that target box, locking on to that target until it is destroyed to lock on to another unit

Challenges faced during the building process of this feature:

A particularly difficult bug involved multiple arrows being fired from a single attack animation. Upon revisiting all my configurations, it turns out that the attack animation had Loop Time enabled within Unity's Animator component. As a result, Unity repeatedly replayed the attack animation before transitioning back to the idle state, causing multiple projectile spawn events to trigger.



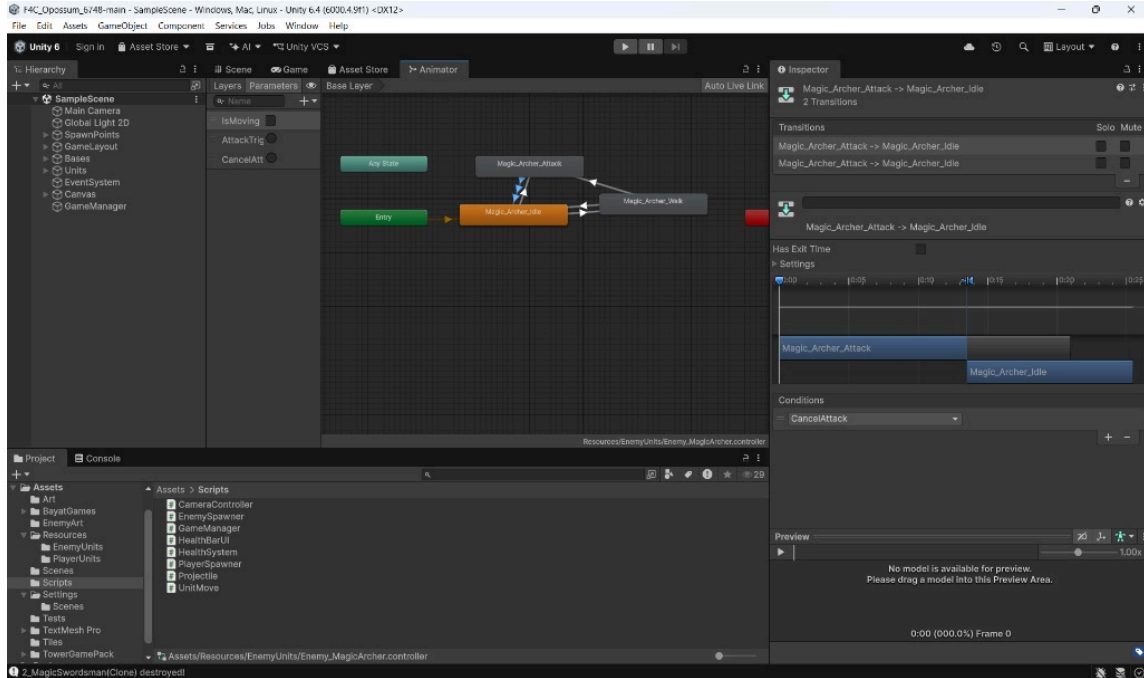
Note: Multiple arrows are being shot from the crossbow turret all at once, even though the console log tracks only one shot should have been fired. In this picture, we are also able to see how the turret attack range is like via the faint yellow box, which is its attackable area.

Disabling Loop Time for the attack animation ensured that only a single animation event was executed during each attack cycle, guaranteeing exactly one projectile per attack.

Other Notable Difficulties Faced Throughout Milestone 2

11.10. Unity Animator State Machine

While integrating the 2D sprites, one of the most significant technical hurdles we faced was understanding and implementing Unity's Animator component. Transitioning from pure C# logic to a visual Finite State Machine (FSM) introduced several unexpected synchronisation issues between our code and the visual output.



- State Machine Complexity:** As seen in the configuration for the `Enemy_MagicArcher.controller`, we had to design a robust flow between Entry, Idle, Walk, and Attack states. Learning how to properly map script variables to Animator Parameters (`IsMoving` boolean, `AttackTrig` trigger, and `CancelAtt` trigger) required extensive trial and error. Some common bugs faced when configuring these settings are the characters being stuck in their idle animations or walking perpetually without attacking enemy units.
- The "Has Exit Time" Problem:** Initially, our units felt sluggish and unresponsive during combat. We discovered this was caused by Unity's default Has Exit Time setting on transitions. Units were waiting for their idle or walk animations to finish 100% of their cycle before transitioning to an attack. Disabling Has Exit Time and relying purely on condition triggers (`AttackTrig`) made combat feel instantly responsive.
- Handling Interrupted States:** An edge case emerged when a target was destroyed while a unit was mid-swing in its attack animation. If the target died, the unit would remain stuck in the attack loop because the FSM lacked an escape route. To solve this, we engineered an emergency interrupt system using the `CancelAttack` trigger. As shown in our Animator setup, this condition forces an immediate transition from the Attack state back to Idle, allowing the `UnitMove.cs` script to safely re-evaluate its next target without animation lock-up.

11.11. Version Control with Unity

This was one of the biggest challenges that we faced as a team, which is not typically encountered in standard web or software development. Managing Git version control with Unity's proprietary file structures caused a massive headache, as Unity scenes (.unity) and prefabs (.prefab) are serialised as massive text files. Early in Milestone 2, concurrent edits to the SampleScene resulted in merge conflicts that corrupted the scene hierarchy. We often had to deal with merge conflicts which resulted in a lot of time wasted pulling, pushing, and figuring out what went wrong and how we can fix it. It had occurred a total of 3 times during Milestone two, and we spent a total of 18 hours fixing these git merging issues.

To resolve this, we took turns working on the main scene and individually, we worked exclusively on isolated Prefabs (e.g., updating the MagicArcher prefab in isolation). This ensured that Unity's Hidden Meta Files setting was explicitly managed, guaranteeing that no missing script references or broken GUIDs occurred during branch merges.

11.12. Conclusion for Milestone 2

Milestone 2 represents a significant progression from the technical proof of concept established during Milestone 1 into a fully playable game prototype. Throughout this development cycle, we successfully integrated the planned economy system, fully animated combat units, modular turret defence mechanics, polished user interface elements, audio management, and seamless scene transitions into a cohesive gameplay experience.

Beyond implementing new gameplay features, a considerable portion of this milestone was dedicated to improving the overall quality and maintainability of the project. Numerous bugs relating to unit movement, animation synchronisation, UI rendering, scene transitions, and asset integration were identified and systematically resolved through extensive gameplay testing and iterative refinement. These debugging efforts not only improved gameplay smoothness but also strengthened our understanding of Unity's animation system, scene management, and software engineering practices.

Overall, we feel that the project has now progressed beyond demonstrating isolated mechanics into a functional prototype that closely resembles the intended final product. The current architecture provides a stable and extensible foundation upon which additional gameplay mechanics and content can be implemented during the final milestone.

11.13. Future Roadmap for Milestone 3

With the core gameplay systems now completed, development during Milestone 3 will primarily focus on expanding game content, improving replayability, and polishing the overall user experience.

Our planned objectives include:

- **Era Progression System:** Expand the current game by introducing additional eras, featuring unique unit rosters, upgraded base appearances, and different animations to better capture the progression mechanics inspired by Age of War.
- **Special Abilities:** Integrate cooldown-based global abilities (e.g., calling down a meteor strike on the battlefield).
- **Advanced AI:** Upgrade the EnemySpawner script to read the player's unit composition and intelligently spawn counter-units, rather than relying on randomised timers.
- **Game Balancing:** Perform extensive balancing of unit costs, movement speeds, attack values, health, turret effectiveness, and economy progression to create a fair and enjoyable gameplay experience.
- **Software Engineering Improvements:** Continue refactoring the existing codebase into smaller, more modular components where appropriate, while conducting comprehensive code reviews, additional gameplay testing, and bug fixing to improve maintainability and overall software quality.

By the completion of Milestone 3, we aim to transform the current prototype into a polished, feature-complete game that delivers a cohesive gameplay experience while demonstrating both the technical and software engineering objectives expected of an Apollo 11 Orbital project.